

Artificial Intelligence (CS303)

Lecture 2: Beyond Classical Search

Hints for this lecture

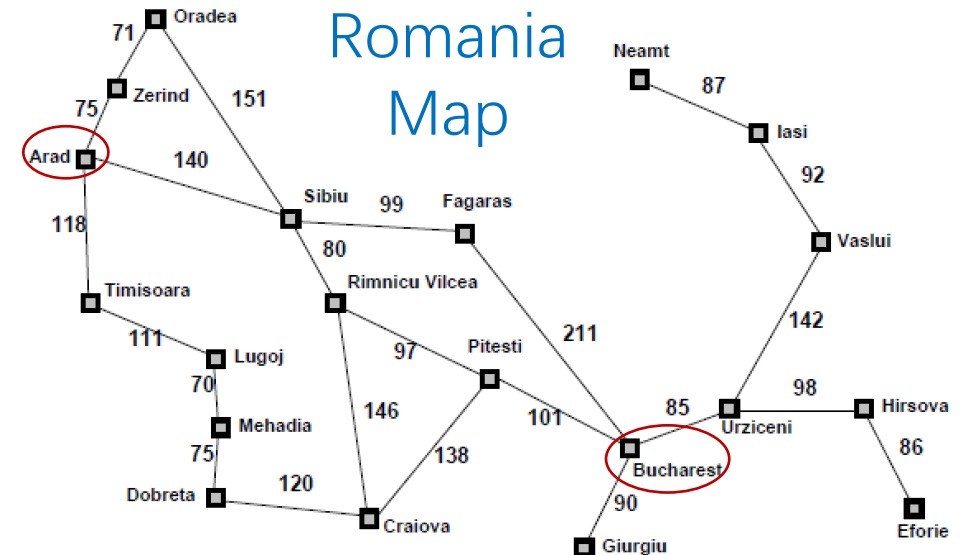
- **“Classical” search: use a tree representation**
- **“Beyond”: generalize to other types of representation**

Outline of this lecture

- More Representations
- General Search Methods
- More complex real-world search scenarios

More representations

- Route planning on Romania map again...
- In lecture 1, we formulated the problem with tree representation.
- Any different way to formulate it?
- In other words, what is a “state”?
- Alternative: permutations of cities
- Which one is better?



More representations

- As long as the state space is finite (finite set), tree representation can be used to formulate any problem.
- Any problem solved on modern computing hardware can be viewed as having a finite state space.
- In many cases, Search Tree is not the most appropriate representation.

More representations

- Consider the ubiquitous optimization problems.



Transportation: Railway timetabling



Energy: Wind turbine design



Architecture: Truss design (Birds nest)



Finance: Portfolio optimization

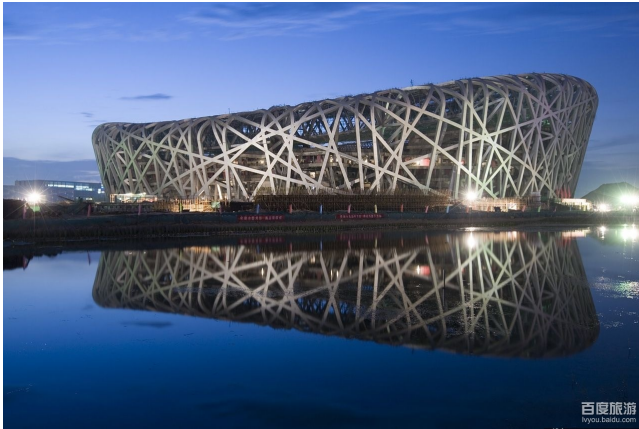
maximize $f(x)$

subject to: $g_i(x) \leq 0, \quad i = 1 \dots m$

$h_j(x) = 0, \quad j = 1 \dots p$

More representations

- How to represent the positions of a set of truss?



Truss Design

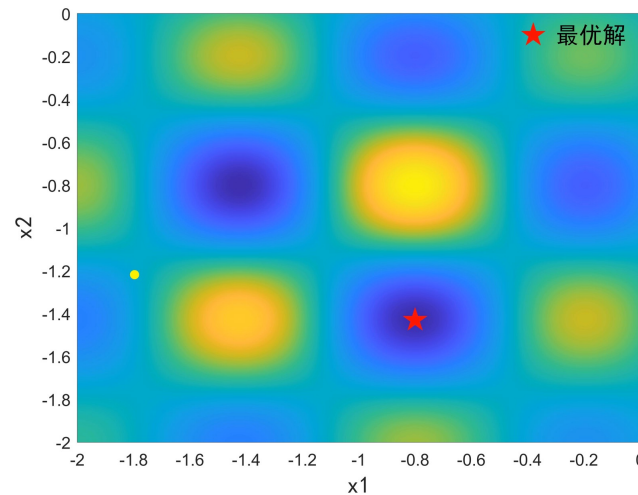
???

maximize $f(x)$
subject to: $g_i(x) \leq 0, \quad i = 1 \dots m$
 $h_j(x) = 0, \quad j = 1 \dots p$

What is the mathematical formulation of these functions?

Direct Search in the Solution Space

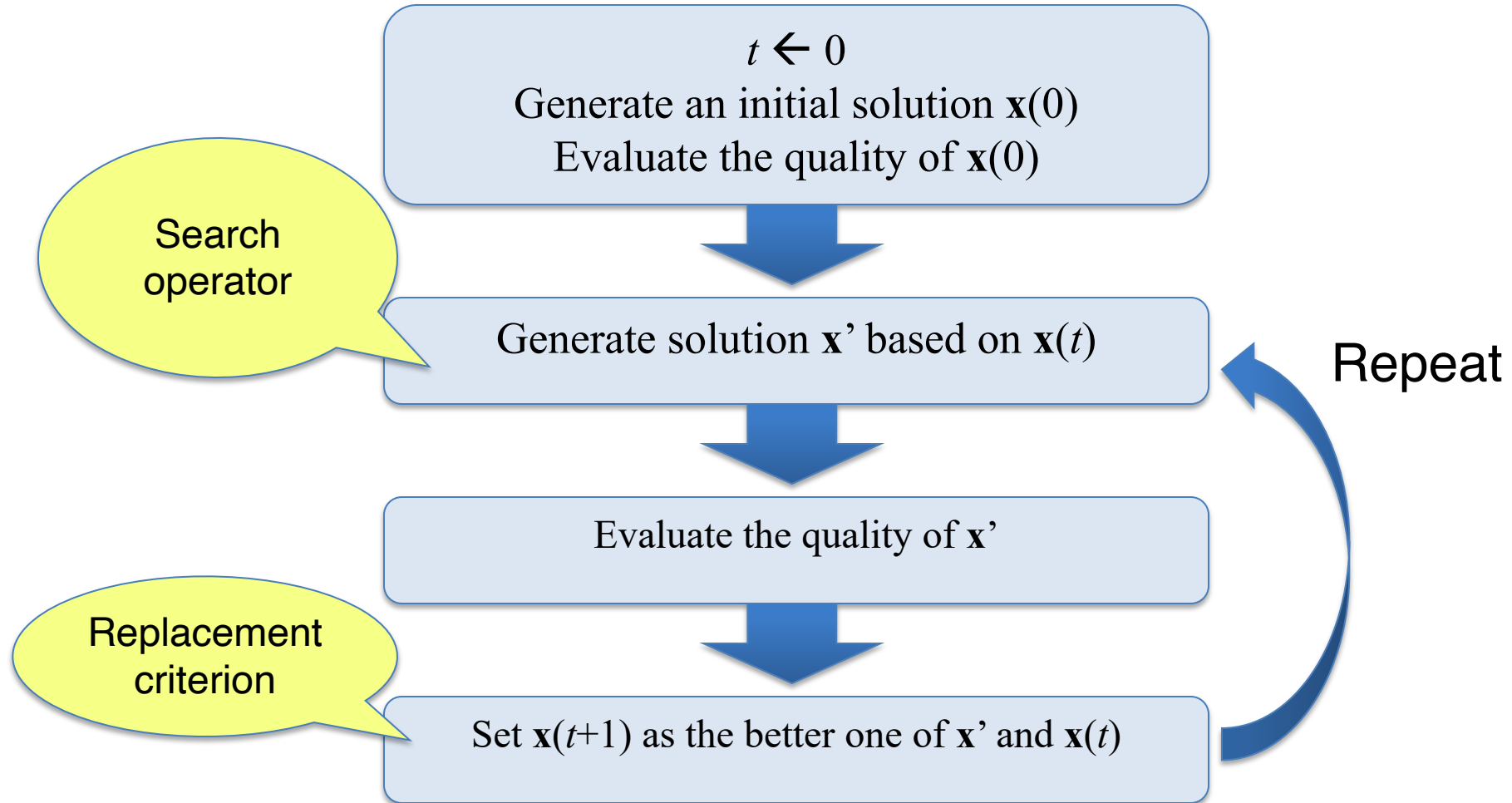
- When tackling an optimization problem, it is more natural to imagine searching for a solution in the **solution space**.
- Each element of the solution space is a **complete solution**.



Direct Search in the Solution Space

- Representations of a solution space can be roughly categorized as:
 - Continuous
 - Discrete
 - Binary
 - Integer
 - Permutation
 - ...
- Different representations may favor different search methods, but most of them share a common framework.

A General Framework for Search



General Search Methods

- **Typical Frameworks:**
 - Local Search
 - Simulated Annealing
 - Tabu Search
 - Population-based search
- **Two basic issues (differs over concrete search methods):**
 - search operator (how to generate a new candidate solution)
 - evaluation criterion (or replacement strategy)

Typical Search Operators

- **A Search Operator generate a new solution based on previous ones.**

$$\phi: x \rightarrow x', \forall x, x' \in \mathcal{X}$$

- **“Un-informed” Search Operators in the solution space: sampling**
 - For a binary string: Flip 1 (or n) bit
 - For a permutation: Exchange the positions of two elements
 - For a continuous number/vector: sample a Gaussian distribution
- **In the rest of this lecture, we take continuous case as an example.**

Greedy Local Search Framework

- **A greedy-type search framework in the solution space**
 - Given a **predefined** Local Search Operator
 - Iteratively generate new solutions
 - Always pick the best solution so far, sometimes also known as Hill Climbing.

Do while (halt condition is not satisfied)

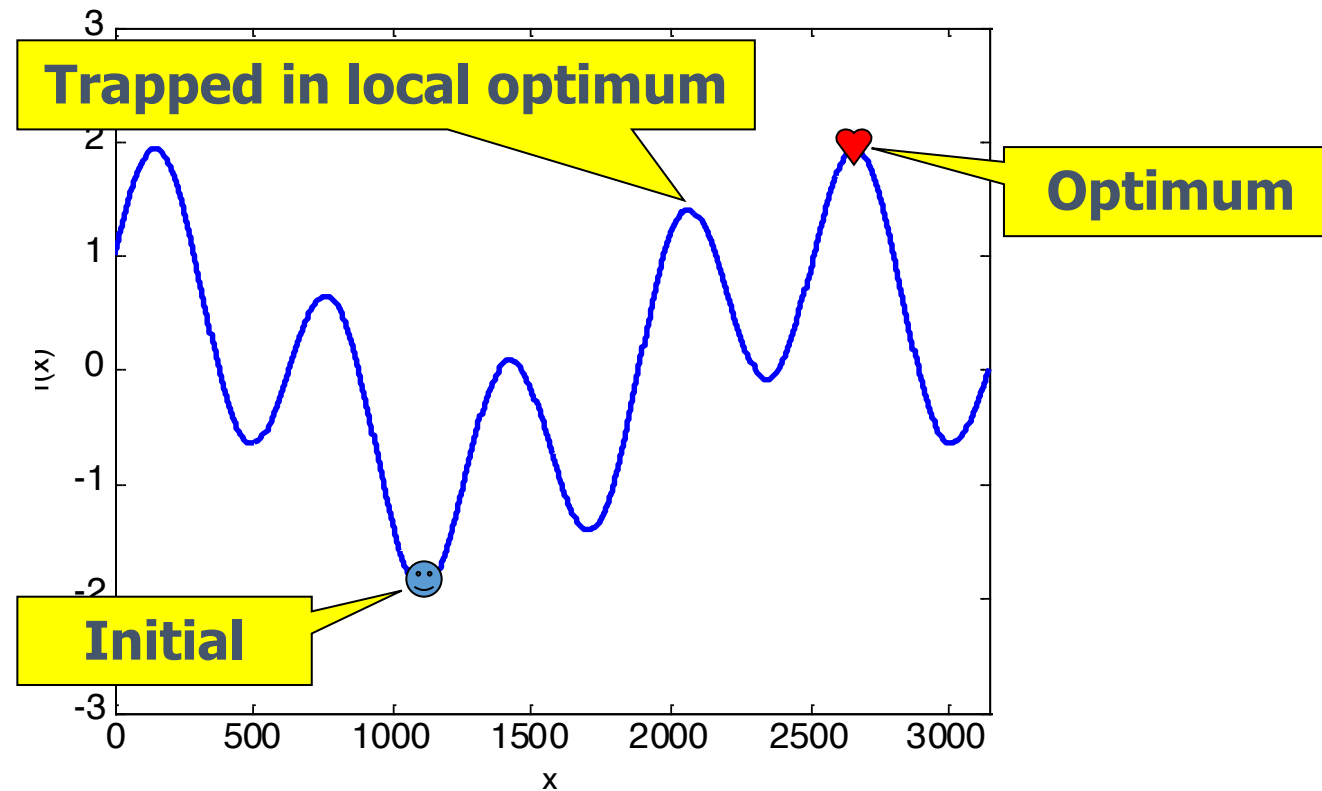
- Generate solution $\mathbf{x}'$ based on $\mathbf{x}(t)$
- Evaluate the $f(\mathbf{x}')$
- If $f(\mathbf{x}') > f(\mathbf{x}_i)$,
- Then $\mathbf{x}(t+1) = \mathbf{x}'$, otherwise $\mathbf{x}(t+1) = \mathbf{x}$
- $i = i + 1$

Greedy~
Hill Climbing~
Steepest Ascent

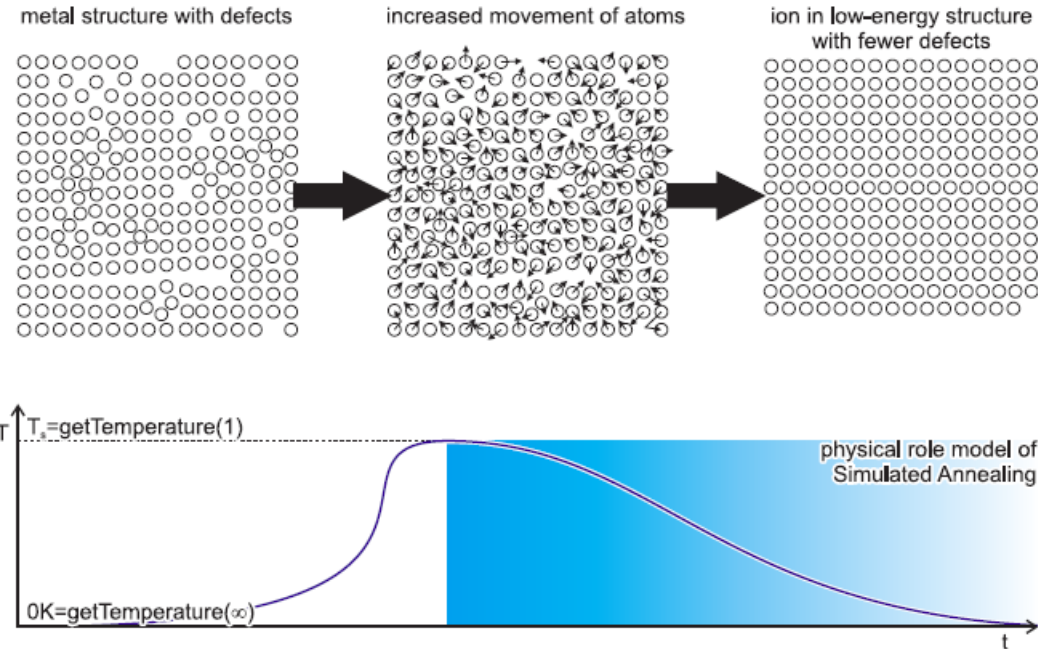
assume a maximization problem

Greedy idea may fail

- On multimodal problems



Simulated Annealing



temperature from high to low
when high temperature, form the shape
when low temperature, polish the detail

Simulated Annealing

- Hill Climbing → Simulated Annealing

Replace this line with
a probability p

Do while (halt condition is not satisfied)

- Generate solution $\mathbf{x}'$ based on $\mathbf{x}(t)$
- Evaluate the $f(\mathbf{x}')$
- If $f(\mathbf{x}') > f(\mathbf{x}_i)$,
- Then $\mathbf{x}(t+1) = \mathbf{x}'$, otherwise $\mathbf{x}(t+1) = \mathbf{x}$
- $i = i + 1$

$$p = \begin{cases} 1 & \text{if } f(x_i) < f(x'_i) \\ \exp\left(-\frac{f(x_i) - f(x'_i)}{T}\right) & \text{if } f(x_i) \geq f(x'_i) \end{cases}$$

Simulated Annealing

- Generate an initial $\mathbf{x}(0)$
- Set the initial temperature $T(0)$;
- Do while (halt condition is not satisfied)
 - Get current temperature $T(i)$;
 - Generate solution $\mathbf{x}'$ based on $\mathbf{x}(i)$
 - Calculate $f(\mathbf{x}')$ and $\Delta f = f(\mathbf{x}(i)) - f(\mathbf{x}')$
 - If $\Delta f < 0$ (for maximization) , then $\mathbf{x}(i+1) = \mathbf{x}'$
 - Otherwise, $p = \exp(-\Delta f / T(i))$;
 - If $c = \text{random}[0,1] < p$, $\mathbf{x}(i+1) = \mathbf{x}'$;
 - Otherwise $\mathbf{x}(i+1) = \mathbf{x}(i)$;
 - $i = i + 1$
 - Update $T(i)$;
- End Do

Other “single-point” Search Framework

- **Hill Climbing → Tabu Search**

- Key idea: Incorporate **MEMORY**. Don't visit the sample candidate solution twice.
- Challenge: How to define the Tabu list?

- **Hill Climbing → Bayesian Optimization**

- Key idea: Build **a model to "guess"** which solution is good.
- Challenge:
 - model building is non-trivial
 - may need lots of data to build the model, limited to low-dimensional problem

Population-based Search

- Since we sample from a probability distribution, why 1 at a time?
- A naive approach: Local Beam Search

Idea: keep k states instead of 1; choose top k of all their successors

Not the same as k searches run in parallel!

Searches that find good states recruit other searches to join them

Problem: quite often, all k states end up on same local hill

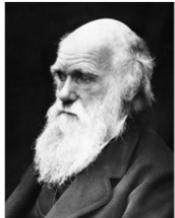
Idea: choose k successors randomly, biased towards good ones

Observe the close analogy to natural selection!

Population-based Search

- Conceptually, a search process that iteratively generates-and-tests a population of solutions is similar to the biological evolution process.

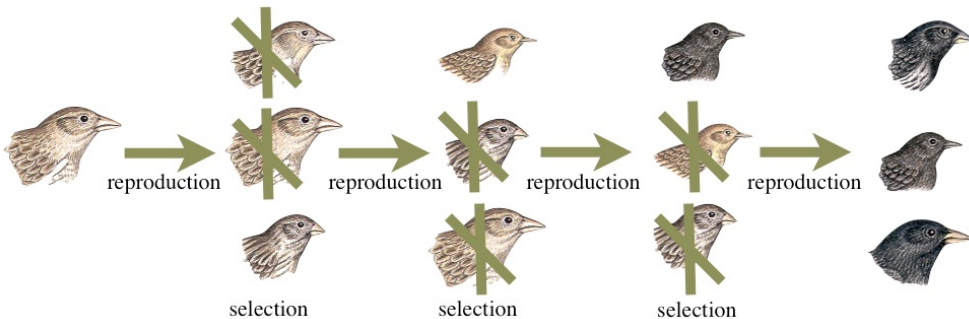
Biological evolution



Charles Darwin
1809-1882

C. Darwin, after collecting abundant evidence, developed a theory about how species evolve.

reproduction with variation + nature selection



Evolution of Programs (1950)



We have thus divided our problem into two parts. The child programme and the education process. These two remain very closely connected. We cannot expect to find a good child machine at the first attempt. One must experiment with teaching one such machine and see how well it learns. One can then try another and see if it is better or worse. **There is an obvious connection between this process and evolution**, by the identifications

Structure of the child machine = hereditary material

A. Turing

Changes of the child machine = mutation,

Natural selection = judgment of the experimenter

One may hope, however, that this process will be more expeditious than evolution. The survival of the fittest is a slow method for measuring advantages. The experimenter, by the exercise of intelligence, should be able to speed it up. Equally important is the fact that he is not restricted to random mutations. If he can trace a cause for some weakness he can probably think of the kind of mutation which will improve it.

Population-based Search

- **Different PBS methods have different names, but more often than not, they are known as Evolutionary Algorithms.**

types of technology.

Representative Technology Categories

The representative general categories of technology for which Commerce currently seeks to determine whether there are specific emerging technologies that are essential to the national security of the United States include:

- (1) Biotechnology, such as:
 - (i) Nanobiology;
 - (ii) Synthetic biology;
 - (iv) Genomic and genetic engineering;
- or
- (v) Neurotech.
- (2) Artificial intelligence (AI) and machine learning technology, such as:
 - (i) Neural networks and deep learning (e.g., brain modelling, time series prediction, classification);
 - (ii) Evolution and genetic computation (e.g., genetic algorithms, genetic programming);
 - (iii) Reinforcement learning;
 - (iv) Computer vision (e.g., object

- (iv) Molecular robotics;
- (v) Robot compliers; or
- (vi) Smart Dust.
- (11) Brain-computer interfaces, such as
 - (i) Neural-controlled interfaces;
 - (ii) Mind-machine interfaces;
 - (iii) Direct neural interfaces; or
 - (iv) Brain-machine interfaces.
- (12) Hypersonics, such as:
 - (i) Flight control algorithms;
 - (ii) Propulsion technologies;
 - (iii) Thermal protection systems; or
 - (iv) Specialized materials (for structures, sensors, etc.).
- (13) Advanced Materials, such as:
 - (i) Adaptive camouflage;
 - (ii) Functional textiles (e.g., advanced fiber and fabric technology); or
 - (iii) Biomaterials.
- (14) Advanced surveillance technologies, such as:
 - Faceprint and voiceprint technologies.

1. Generate the initial **population** $P(0)$ at random, and set $i \leftarrow 0$;
2. REPEAT
 - (a) Evaluate the fitness of each individual in $P(i)$;
 - (b) **Select** parents from $P(i)$ based on their fitness in $P(i)$;
 - (c) **Generate** offspring from the parents using *crossover* and *mutation* to form $P(i + 1)$;
 - (d) $i \leftarrow i + 1$;
3. UNTIL halting criteria are satisfied

Population-based Search: A Concrete Example

- Set $i=0$; Population_size = N
- Generate N initial solutions, $\mathbf{X}(0)$;
- Do while (halt condition is not satisfied)
 - Evaluate the quality of the solutions in $\mathbf{X}(i)$; (% calculate $f(x)$ for each solution)
 - Use the **top k** solutions in $\mathbf{X}(i)$ to build a **Gaussian Distribution P_i**
 - Sample another N according to P_i , as $\mathbf{X}(i+1)$
 - $i=i+1$
- End Do

The example could be viewed as searching for a **good probability distribution** (parameters of the Probability Density Function), from which good solutions could be sampled.

Why “Population” could be Beneficial?

- Seeking a good distribution: maximize the following “objective function”

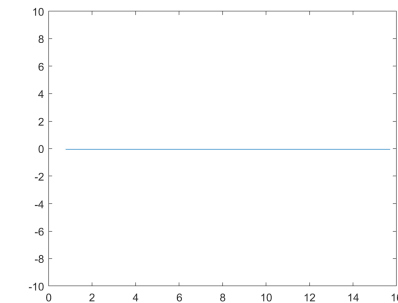
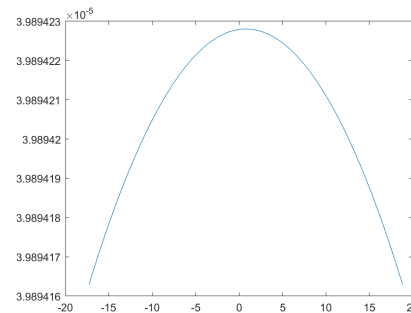
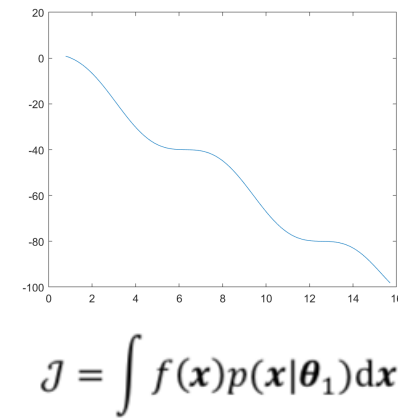
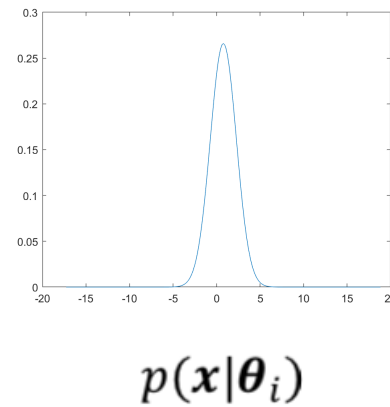
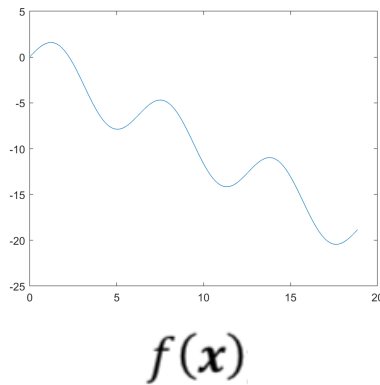
$$\mathcal{J} = \int f(\mathbf{x}) p(\mathbf{x}|\boldsymbol{\theta}_1) d\mathbf{x}$$

probability density function parameterized by θ_1

- One advantage for such a transformation (and thus for using a population): the original objective function is “smoothed”.

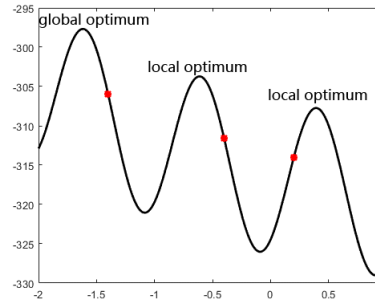
Why “Population” could be Beneficial?

- The smoothing effect
- Beneficial with **appropriate** hyper-parameters, e.g., variance of a Gaussian distribution



Making Population Even More Beneficial?

- Suppose the algorithm has converged to some (global or local) optimum



- If run the algorithm again, we hope the algorithm (i.e., the distribution corresponding to the final population) converge to a **different** optimum (and thus a different PDF).

$$\mathcal{J} = \int f(x)p(x|\theta_1)dx + \int f(x)p(x|\theta_2)dx + (-C(\theta_1, \theta_2) - C(\theta_2, \theta_1))$$

$$\mathcal{J} = \sum_{i=1}^{\lambda} \int f(x)p(x|\theta_i)dx + \sum_{i=1}^{\lambda} \sum_{j=1}^{\lambda} (-C(\theta_i, \theta_j))$$

similarity of the two PDFs

More Examples of Evolutionary Algorithms

Let's use the simple EA to maximise the function $f(x) = x^2$ with x in the *integer* interval $[0, 31]$, i.e., $x = 0, 1, \dots, 30, 31$.

The first step of EA applications is *encoding* (i.e., the representation of chromosomes). We adopt binary representation for integers. Five bits are used to represent integers up to 31. Assume that the population size is 4.

1. Generate initial population at random, e.g., 01101, 11000, 01000, 10011. These are *chromosomes* or *genotypes*.
2. Calculate fitness value for each individual.
 - (a) Decode the individual into an integer (called *phenotypes*),

$$01101 \rightarrow 13, 11000 \rightarrow 24, 01000 \rightarrow 8, 10011 \rightarrow 19;$$

More Examples of Evolutionary Algorithms

(b) Evaluate the fitness according to $f(x) = x^2$,

$$13 \rightarrow 169, 24 \rightarrow 576, 8 \rightarrow 64, 19 \rightarrow 361.$$

3. Select two individuals for crossover based on their fitness. If roulette-wheel selection is used, then

$$p_i = \frac{f_i}{\sum_j f_j}.$$

Two offspring are often produced and added to an intermediate population. Repeat this step until the intermediate population is filled. In our example,

$$p_1(13) = 169/1170 = 0.14 \quad p_2(24) = 576/1170 = 0.49$$

$$p_3(8) = 64/1170 = 0.06 \quad p_4(19) = 361/1170 = 0.31$$

Assume we have *crossover*(01101, 11000) and *crossover*(10011, 11000). We may obtain offspring 0110 0 and

More Examples of Evolutionary Algorithms

1100 1 from *crossover*(01101, 11000) by choosing a random crossover point at 4, and obtain 10 000 and 11 011 from *crossover*(10011, 11000) by choosing a random crossover point at 2. Now the intermediate population is

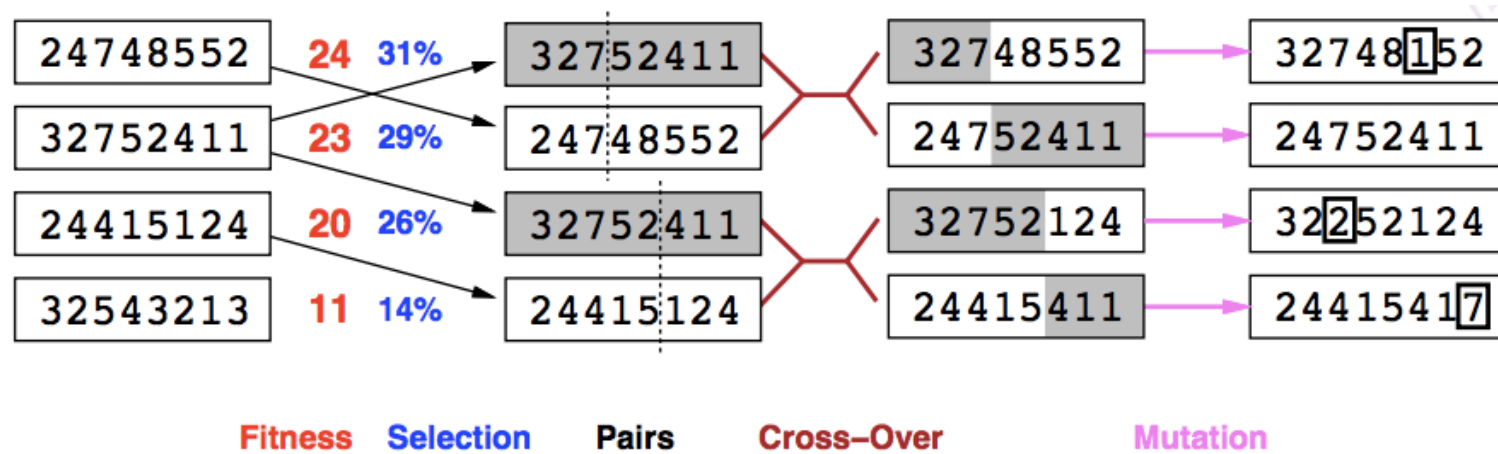
01100, 11001, 10000, 11011

4. Apply mutation to individuals in the intermediate population with a *small* probability. A simple mutation is bit-flipping. For example, we may have the following new population $P(1)$ after random mutation:

0110**1**, 11001, **0**0000, 11011

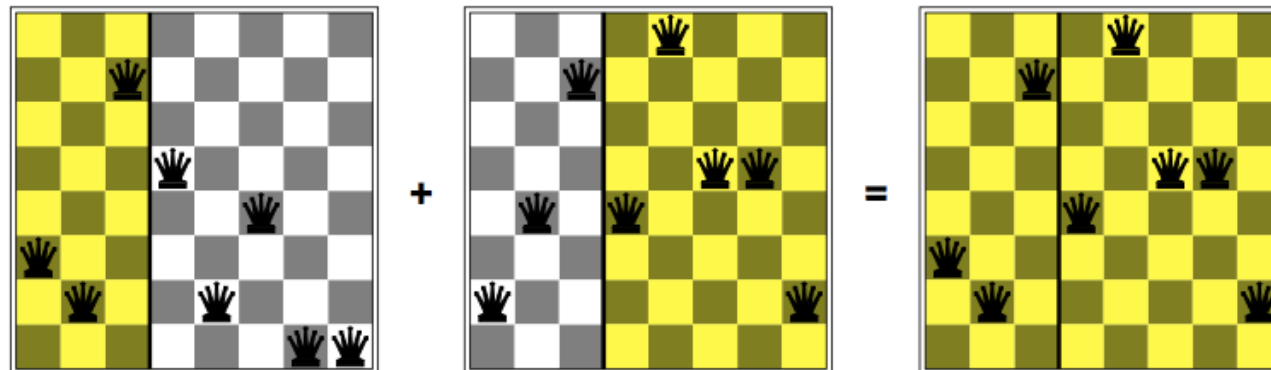
5. Goto Step 2 if not stop.

More Examples of Evolutionary Algorithms



GAs require states encoded as strings (GPs use programs)

Crossover helps **iff substrings are meaningful components**



Some applications of EAs

- High-speed train head design (Japan)



Series 700 Designed by human

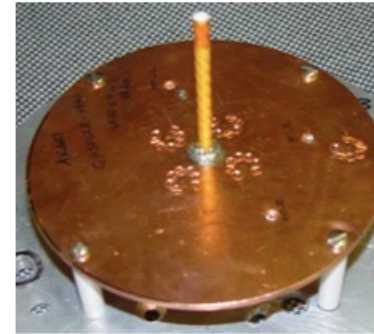
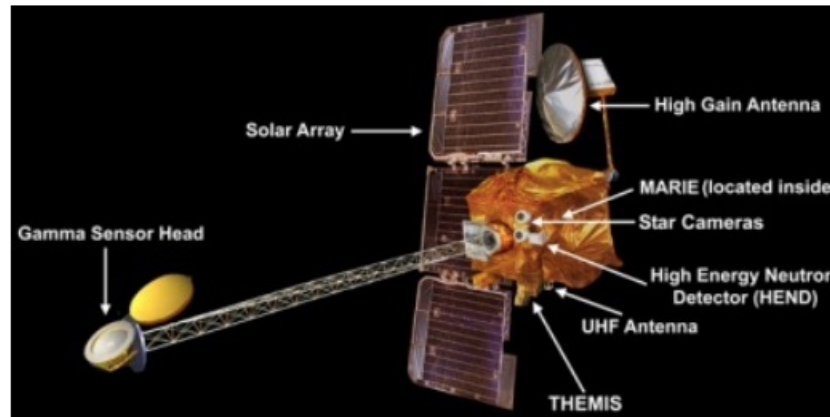


Series N700 Designed by EA

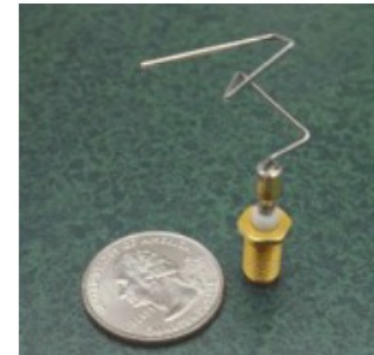
- Save 19% energy...30 increase in the output...

Some applications of EAs

- X-Band Antenna Design (NASA, US)



Human

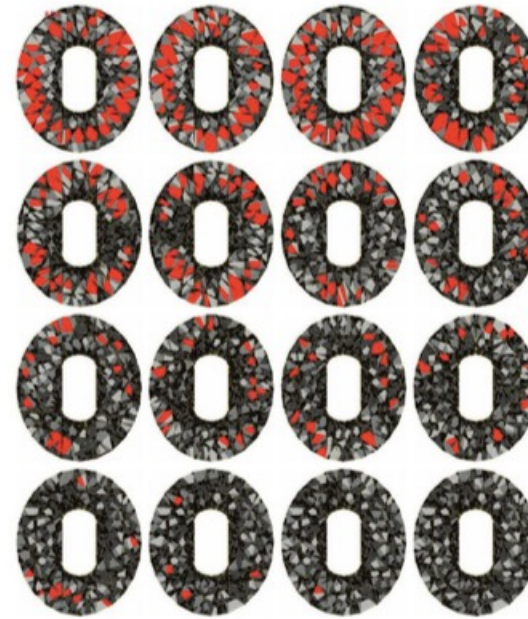


EA

- Increase efficiency from 38% to 93%

Some applications of EAs

- Birds Nest (China & Switzerland)



- The irregular ordering of the beams poses an insoluble problem for the then-current CAD tools.

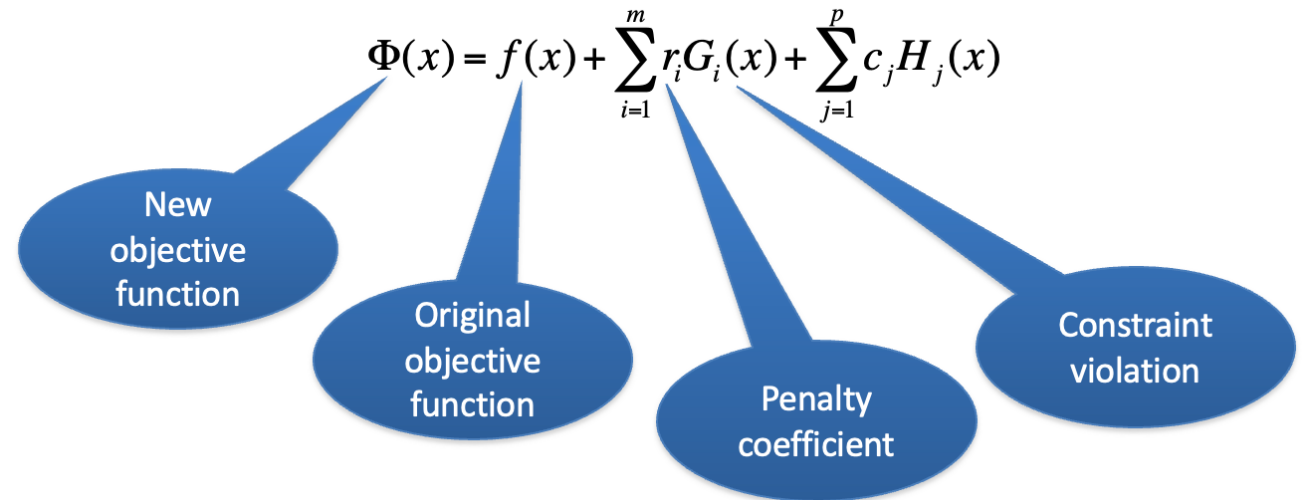
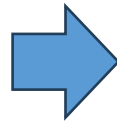
Extended Information – Constrained Optimization

Most realistic problems are subject to constraints

- $f(x)$: the objective function
- $g_i(x)$: the inequality constraints
- $h_j(x)$: the equality constraints

But they could be transformed into unconstrained problems.

minimize $f(x)$
subject to: $g_i(x) \leq 0, \quad i = 1 \dots m$
 $h_j(x) = 0, \quad j = 1 \dots p$

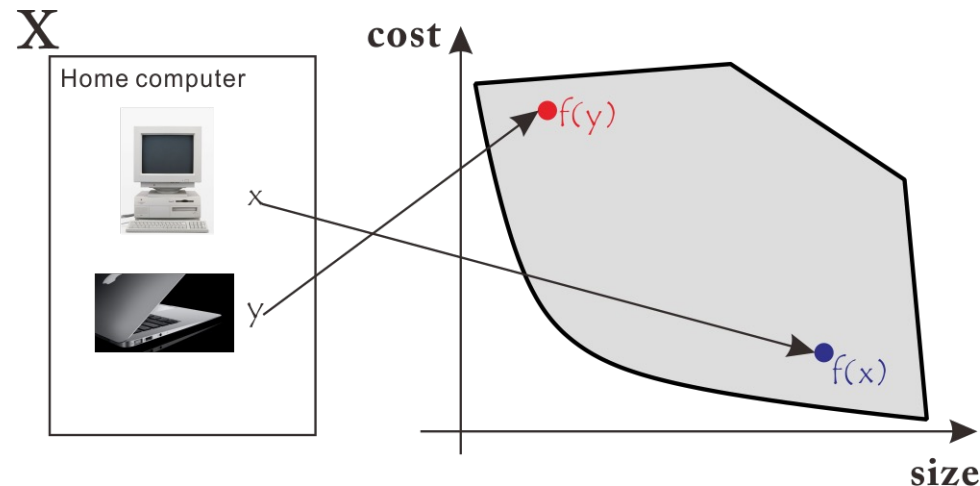


Extended Information – Multi-Objective Optimization

- Multi-objective optimization problem:

$$\text{minimize } F(X) = (f_1(x), \dots, f_m(x))$$

- The objective functions may be conflicting or incommensurable.
- Instead of a single optimal solution, **a set of the best trade-off solutions** (Pareto optimal solutions) are sought.

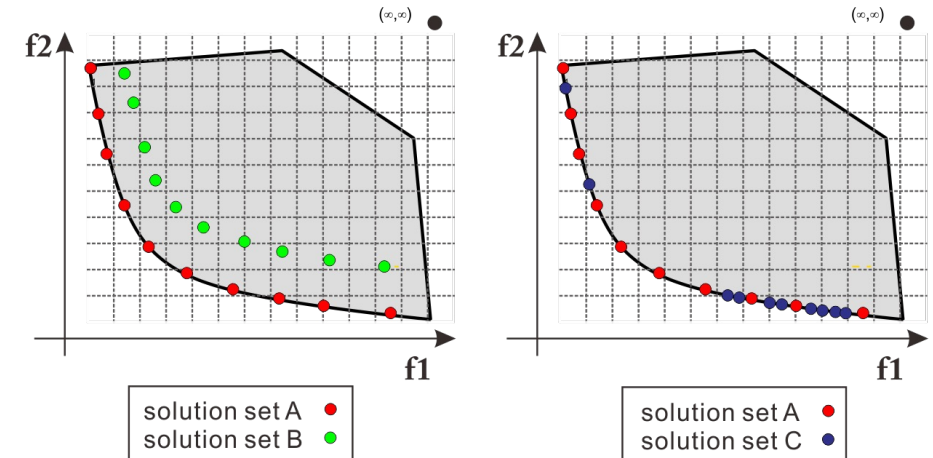


Extended Information – Multi-Objective Optimization

Multi-objective optimization itself involves multiple criteria.

Intuitively, we want a set of solutions that is good in terms of:

- Convergence (to the Pareto front)
- Spread (along the Pareto front)



You can also transform them into single-objective problems, or

Use population-based search

- If interested, check the Non-dominated Sorting Genetic Algorithm II paper published in 2002 to start.
- Or consult with Prof. Hisao Ishibuchi.

Remarks

- All search methods talked in this lecture are **general-purpose search frameworks** (like A^*), rather than search algorithms. Each search frameworks encompasses many concrete instantiations (algorithms).
- When addressing a specific problem
 - You can use the off-the-shelf version, or
 - heuristics derived from domain knowledge could be incorporated in forms of search operators to obtain better performance.

Remarks

- A quick google search would direct you to some of the off-the-shelf versions, e.g.,
 - Greedy Algorithm (hill-climbing for 0-1 optimization problem)
 - Genetic Algorithm (mostly for 0-1 problem)
 - Bayesian Optimization Algorithm (for continuous problem)
 - Covariance-Matrix Adaptation Evolution Strategies (for continuous problem)

To be continued